

浙江大学



水下机器人设计 课程期末报告

姓名: 金之杭 胡蒙奇 郑思塬

学院: 海洋学院

专业: 海洋工程与技术

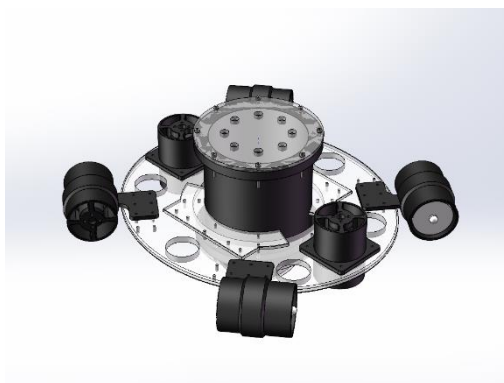
学号: _____

指导教师: 陈正 刘勋

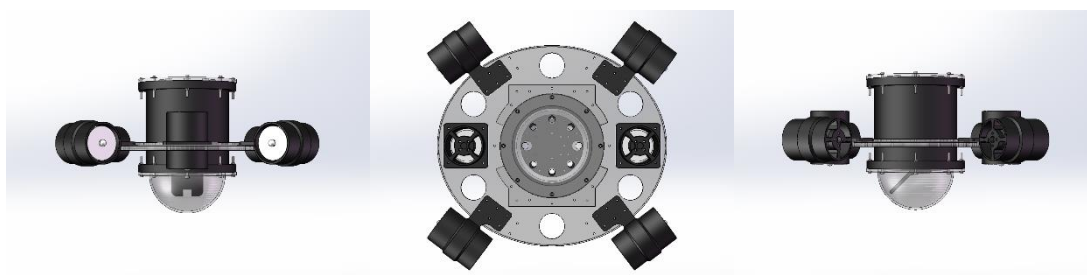
日期: 2024.6.19

1.机械设计

水下机器人需实现的首要功能是运动功能，运动需要控制水下机器人的俯仰角、横摇角、偏航角、进退、横移和沉浮等六个自由度。拟考虑一种圆碟形六推进器的水下机器人，可以实现良好的推进和操作性能，并且易于加工、组装和调试。



如图所示，本文所述圆碟形水下机器人，围绕圆碟形龙骨展开设计，耐压舱和推进器等器件均使用可灵活调整的连接件连接至龙骨上。



龙骨为椭圆亚克力板件，中间一处大开孔用于放置耐压舱，前后较大开孔用于放置垂直推进器，其余小孔用于水平推进器和其他连接件的连接。推进器连接件均由 3D 打印方式加工，分为竖直和水平。舱内安装方案参考极限智控舱内方案，固定板、控制板、摄像头支架依次放置。摄像头支架有不同角度版本，放置在舱体内部最下方。耐压舱和推进器均使用极限智控产品，减轻设计和加工压力。

2.电路部分

2.1 电路设计思路

水下机器人需要完成图像检测和水下运动两方面的功能。其中，在图像检测方面，采用树莓派+摄像头的方案，实现对黑色铝型材的识别巡线与海洋生物图片的识别。在水下运动方面，需要使用 STM32 微控制器接收水深、姿态、巡线方向信息，控制电机带动螺旋桨，推动水下机器人前进。

首先，电路部分最重要的是电源系统，负责为整个机器人提供能量。续航应满足约为 30min，总电流预估为 9A，估算使用 6000mAh 左右的电池。

传感器系统为水下机器人提供传感信号，用于反馈控制与比赛巡线、识别，主要需要姿态传感器、深度计和水下摄像头。

推进系统为机器人的运动产生推力、扭矩，主要包括推进器与驱动器，为保证水下速度，采用水平四推进器布局，并采用垂直两推进器布局控制俯仰角。

嵌入式系统是水下机器人的核心运算平台，其需要接收传感器系统传出的信号并进行数据处理，以此得出机器人的运动状态及期望运动状态，并驱动电机完成运动。

通讯系统为方便水下机器人调试、检测运行状态、数据传输的辅助系统。

2.2 电路器件选型

在电源系统方面，向商家定制 6400mAh 四串锂电池，为机器人提供能量。电源管理功能由所选小笼包 XLB 控制板实现，包括推进器启停和电源分配功能。

在传感器系统方面，深度计 MS5837 使用 I2C 接口，理论精度可达 2mm，极限深度为 200m，可用于水下机器人的定深航行。具体型号为极限智控的抗干扰型 MS5837-30B。选用维特智能的 JY901S 九轴姿态传感器，可检测 3 轴加速度、3 轴角速度和 3 轴磁场，能对水下机器人的运动实现闭环精确控制。摄像头方面，选用 720p 无畸变摄像头，可在水下拍摄清晰图像。

推进系统方面，采用 6 个极限智控生产的 T50-L20 推进器并采用 ESC20-4 电子调速器驱动，可产生较好的线性推力。

嵌入式系统方面，stm32 部分选用极限智控生产的 XLB 控制板，主控芯片为 STM32L431RCT6 芯片，由开关使能，其接口明细表如下，可完全适配水下机器人的使用要求。该控制板可以极大降低开发难度，减少初期试错成本，为团队后期调试争取了较多时间。

控制板接口明细表

序号	类型	名称	STM32 引脚	树莓派引脚	备注
1	开关接口	弱电开关	——	——	高于 7V 触发
2	检测接口	系统状态	PB1	——	正常高电平
3	使能接口	大功率电源	PB0	——	高电平触发
4		树莓派电源	PB2	——	高电平触发
5		LED0	PC6	——	高电平触发
6		LED1	PC7	——	高电平触发
7		LED2	——	BCM5	高电平触发
8		LED3	——	BCM6	高电平触发
9		蜂鸣器	PB12	——	高电平触发
10	IMU 接口	IMU_RXD	PB6	——	USART1_TX
11		IMU_TXD	PB7	——	USART1_RX
12	深度计	SDA	PB11	——	I2C2_SDA
13		SCL	PB13	——	I2C2_SCL
14	电调接口	MOTOR0	PA8	——	TIM1_CH1
15		MOTOR1	PA9	——	TIM1_CH2
16		MOTOR2	PA10	——	TIM1_CH3
17		MOTOR3	PA11	——	TIM1_CH4
18		MOTOR4	PA5	——	TIM2_CH1
19		MOTOR5	PB3	——	TIM2_CH2
20	LORA 接口	LORA_M0	PA4	——	——
21		LORA_M1	PA6	——	——
22		LORA_RXD	PA2	——	USART2_TX
23		LORA_TXD	PA15	——	USART2_RX
24		LORA_AUX	PA7	——	——
25	树莓派	USART3_TX	PC4	BCM15	树莓派 RXD
26		USART3_RX	PC5	BCM14	树莓派 TXD

同时选用树莓派 5 作为协处理器，进行图像处理。

通讯系统方面，选用 LORA 无线串口作为信息传输工具。

3.控制部分

使用 STM32 裸机编程，利用无限循环与中断配合完成控制任务。

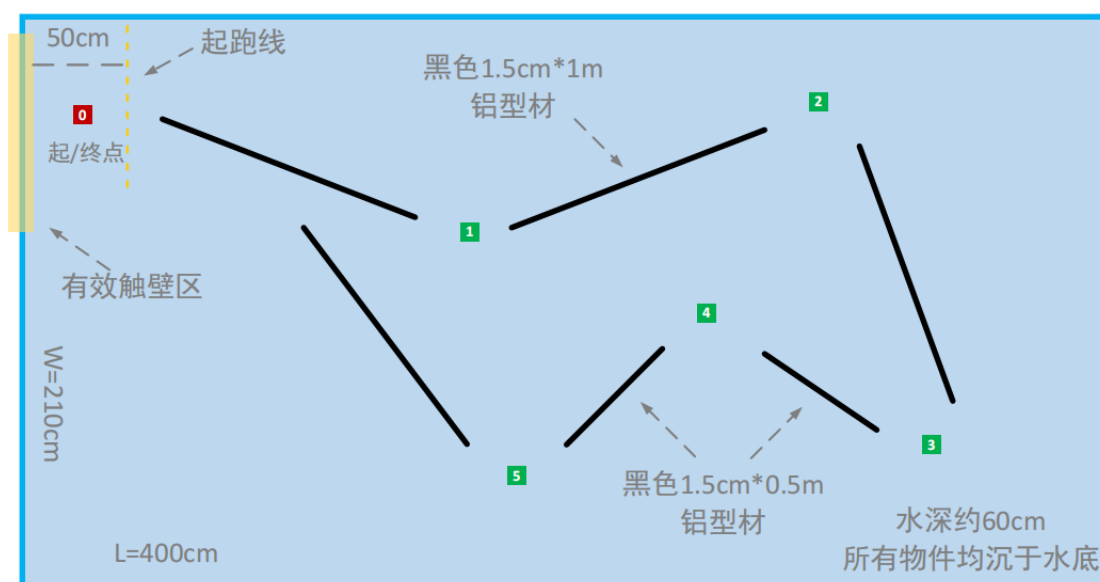
无限循环中，读取深度计数据、运行 PID 得出控制指令并写入定时器。

各个中断负责读取 IMU 数据并运算、读取串口控制指令并执行、读取树莓派返回的巡线数据。

4.视觉部分

本项目的视觉处理方案基于 Opencv-Python 和 YOLO v5 Lite，实现巡线、识别两个功能。同时为了配合 STM32 的控制，设计通信模块。该三个模块采用多进程方法进行配合，并针对比赛所需要的效果进行优化，即可实现视觉部分的功能。

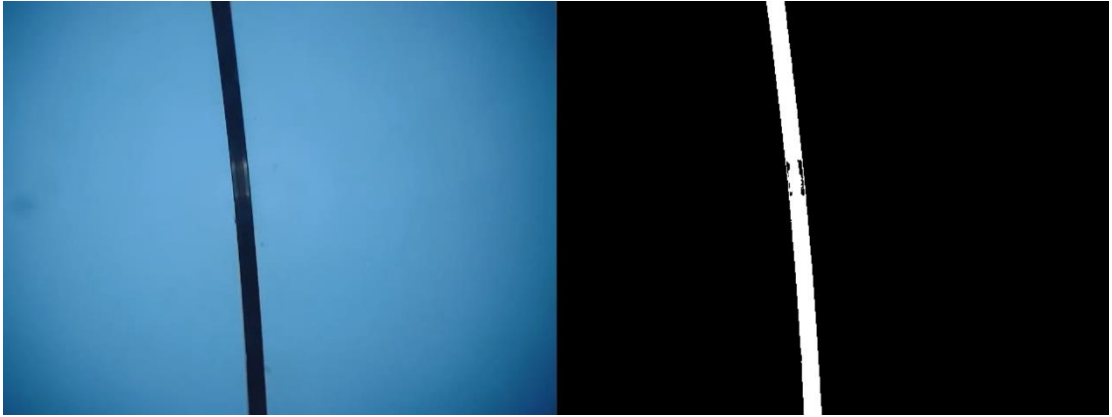
4.1 巡线模块



赛场水池布置图

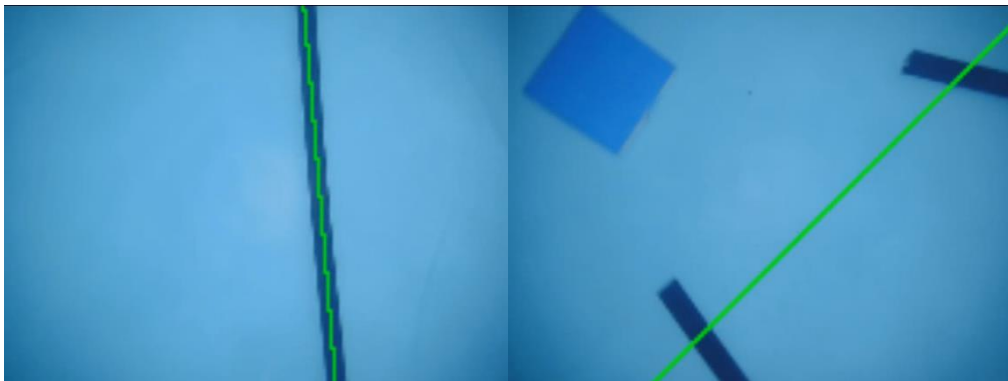
本次竞赛使用黑色铝型材作为视觉导航辅助，故采用巡线方法进行导航。

不管采用哪种巡线算法，第一步都是通过一定手段，二值化图像后提取直线所在像素。由于铝型材和池底颜色区别明显，故根据图片 HSV 空间内各通道特点设置合适的阈值，通过 cv2.inRange 函数制造掩膜，并筛选出直线。



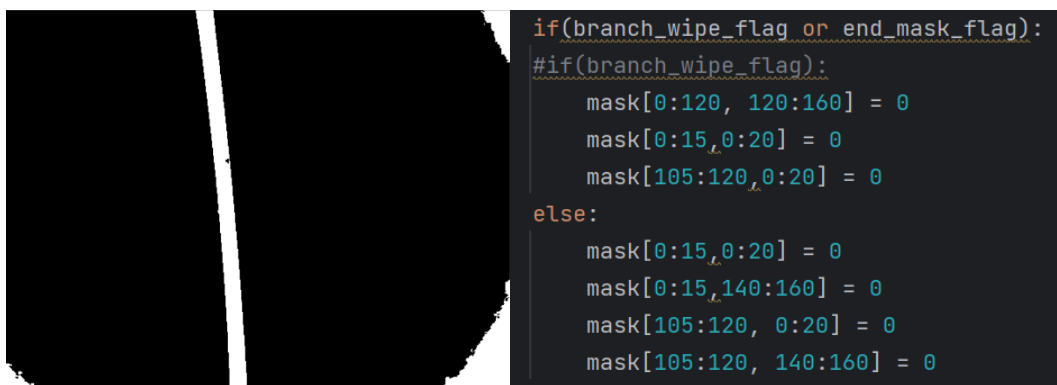
原图和 inRange 函数处理过后的图片

在巡线技术路线的选择上，我们选择了最小二乘法拟合直线，而不是更主流的霍夫直线检测算法。尽管霍夫直线算法对算力的占用量较小，但是其容易受到噪声干扰和线段形状的干扰，特别是本次比赛识别目标海洋生物卡片内有大量杂乱线条，对霍夫直线算法的干扰更甚，会导致机器人控制卡顿、振荡甚至偏离线路。最小二乘法拟合不仅抗噪声性能好，在转弯处也可以连续过渡，甚至图像的存在能使过渡更平滑，有利于控制稳定。



直线和转弯处最小二乘法拟合直线

在调试巡线算法过程中，容易受到暗角影响。由于机器人自身会在水池投下影子，摄像头拍摄水下图片由内而外形成颜色逐渐加深的阴影圈，四角颜色最深，有时颜色会接近铝型材导致难以通过 HSV 方法过滤，从而使暗角像素影响拟合直线的准确度，导致机器人偏离既有线路。我们的解决方法是在四角手动添加遮罩，去除过滤出的暗角像素。

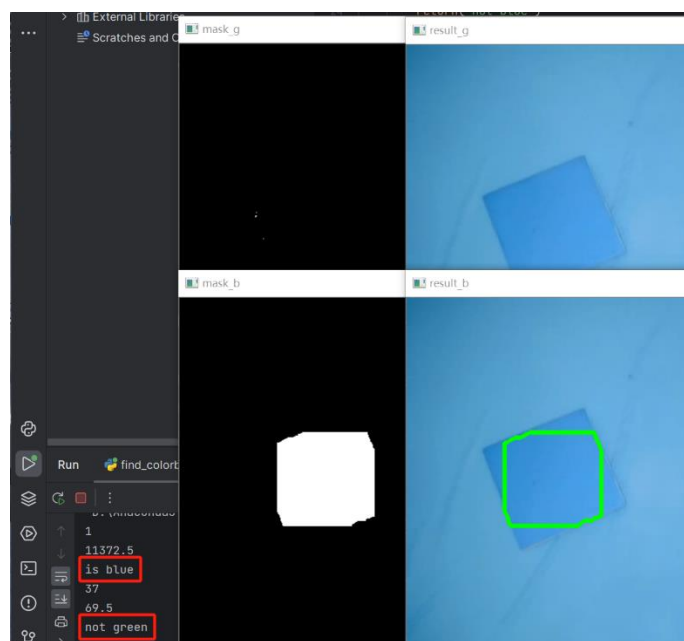


掩膜出现暗角及手动遮罩程序

4.2 图像识别模块

本次比赛图像识别内容是海洋动物图像卡片和 RGB 纯色卡片，对于这两类卡片我们采用了不同的识别方案。

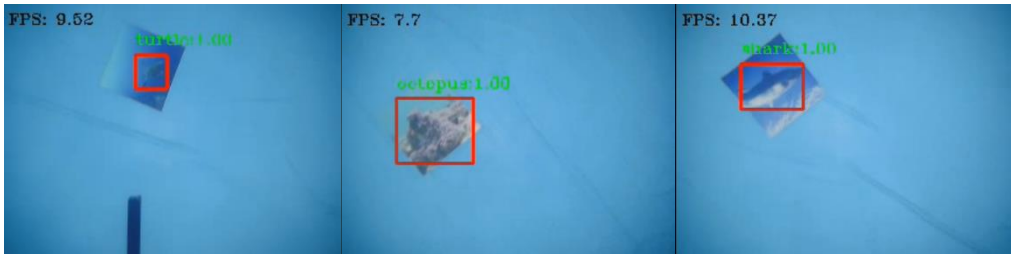
对于 RGB 纯色卡片，我们的思路也是利用 HSV 空间和 cv2.inRange 函数构造掩膜，在此基础上进行开操作过滤噪声，然后提取图中轮廓并保留最大的一个。如果最大轮廓围成面积大于阈值，则判定为识别到纯色卡，返回识别结果。



HSV 方法的纯色卡识别

对于海洋动物图像卡片，我们原本试图采用模板匹配或支持向量机等方法进行识别，但是由于卡片背景颜色的蓝色和池底蓝色接近，并且图内动物大小数量不一，难以截取出固定区域大小的图像用于训练和识别，因此我们转而采用了 YOLO 进行识别。我们在水底固定高度处拍摄大量海洋动物图像照片作为数据集训练 YOLO 模型，训练效果较好，识别的准确率

和稳定度均维持在较高水平。之后我们将 YOLO 识别函数移植到代码内，返回识别结果用于保存图像、闪灯等后续操作。



YOLO 识别海洋生物卡片

在确定识别到图片后，将当前图片保存至树莓派本地，并且向通信模块和闪灯程序发送信号，执行对应的闪灯和回传操作。



拍摄图片按照‘点位+识别物’命名并保存到本地

在完善图像识别模块的过程中，我们遇到了一下几个问题：

1.YOLO 运算量过大，识别速度慢。为了摆脱 Python 全局解释器锁的影响，程序从原本的多线程改为多进程，为图像识别单独开启一个进程，充分利用树莓派多核算力，大大提高了 YOLO 的识别速度。

2.水池光照条件多变，影响动物卡片识别效果。我们在不同光照条件下拍摄不同的数据集，并将它们混合起来用于训练，能够提高识别结果的鲁棒性。

4.3 串口通讯模块

本次比赛题目中要求每识别到一张图片，机器人要为电脑回传识别结果，因此树莓派要利用串口发送识别结果给 STM32。同时巡线部分也要不间断发送直线数据至 STM32。最开始两个模块各自独立发送数据，但是在调试过程中发现这会导致两个模块对串口资源的抢夺，造成通讯故障。因此我们单独设计了串口通讯模块，统一处理串口的收发操作。在每次轮询不同模块的发送需求时，巡线模块每次都要发送数据，而图像识别模块只有在识别到新图像的时候才需要发送数据，其余时间则跳过。

```
# 子进程启动
mainProcess = Process(target=MainProcess, args=(pic_recg, camera,turn_on,linefollow_result,mes_flag_q, Recg_start,end_mask, ))
imgRecg = Process(target=RecProcess, args=(pic_recg,recg_result, Recg_start, ), daemon=True)
serialThread = Process(target=serialRS, args=(linefollow_result, recg_result, turn_on,mes_flag_q,end_mask, ), daemon=True)
```


启动三个进程的代码图

为了辨别发送数据的含义，我们单独设计了通讯协议，数据头为 'L' 时表明发送巡线数据，数据头为 'R' 时表明发送识别结果。

Raspberry Pi --> STM32

- 数据头：无
 - 循线结果: `Raspi_rx_str[0] == 'L'`
 - 数据结构: "%f %f", 偏离中线角度(-90~+90), 半高度处交点横坐标(0~160)
 - 识别结果: `Raspi_rx_str[0] == 'R'`
- 数据尾: `Raspi_rx_str[end] == 0x0A && Raspi_rx_str[end-1] == 0x0D`

树莓派向 STM32 传输数据的通讯协议

5.成果

获得第八届浙江省大学生机器人竞赛水下机器人比赛冠军、一等奖。

